

WEEK 6 SUMMARY

Sprint 1: Database Design, EF Core Modeling, Core Routes, and Transactional Business Logic

- Program:** Backend Development Internship (.NET)
Week: Week 6 of 10 | Phase 3: Capstone Project Development
Project: Cardiac Patient Monitoring API
Theme: Sprint 1 - schema, catalog routes, medication ordering, and sprint close-out
Duration: 40 hours across five training days

Executive Summary

Week 6 marked the first implementation sprint of the capstone project. The work moved from planning and schema design into a complete EF Core model, migration-backed SQL Server structure, production-conscious read endpoints, and a medication-order workflow with real business rules. The sprint used the existing Cardiac Monitor API as its foundation and extended it without exposing database entities directly through the public contract.

The central technical outcome was an end-to-end flow that can browse patients through a paginated, filterable, and sortable DTO endpoint, then create a medication order that validates stock, calculates line and order totals, stores price history, and decrements inventory atomically. Day 5 converted these features into review evidence through automated tests, a repeatable Postman collection, an updated ERD, a sprint backlog, and a concrete retrospective action.

Learning Objectives Achieved

- Planned Sprint 1 around a written goal and a set of sized, verifiable backlog items.
- Documented the Cardiac Monitor domain and its relationships in a Chen-style ERD.
- Configured EF Core relationships, precision, constraints, delete behaviors, seed data, and migrations.
- Built a patient catalog endpoint with pagination, name and gender filters, sorting, and DTO projection.
- Implemented medication-order creation with stock checks, calculated totals, and a serializable transaction.
- Prepared a seven-request Postman demo and expanded the automated suite to 27 passing tests.

Five-Day Curriculum and Practical Work

Day	Focus	Key Outcome
1	Sprint planning and database design	Sprint goal, backlog, and complete domain ERD.
2	EF Core model and migrations	Explicit relationships, seed data, constraints, and migration history.

Day	Focus	Key Outcome
3	Catalog and read operations	Paginated, filterable, sortable patient DTO endpoint.
4	Write operations and business logic	Transactional medication order with stock and totals.
5	Sprint Review and retrospective	Postman demo, acceptance review, backlog, and Sprint 2 action.

Day 1: Sprint Planning and Full Schema Design

Sprint Planning began by defining Done for the week: the project needed a stable database model, current migrations, a reviewable catalog endpoint, and a complete medication-order flow. Tasks were divided into units small enough to finish and verify within roughly half a day to one day, preventing the sprint from becoming an untracked collection of unrelated changes.

The schema was organized around Patients, VitalSigns, Medications, Appointments, MedicationOrders, and MedicationOrderItems, with ASP.NET Core Identity users providing patient ownership and doctor references. The presentation ERD focuses on the business domain; framework role and refresh-token tables remain in the database model but are intentionally omitted from the visual diagram.

Hands-on result: Finalized the Sprint 1 goal, sized backlog, domain relationships, primary and foreign keys, cardinalities, and the Chen-style ERD used in the mentor handoff.

Day 2: EF Core Data Model and Migrations

Every business entity was represented as a C# class with navigation properties matching the ERD. AppDbContext uses the Fluent API to make relationships and delete behavior explicit: patient clinical records cascade where appropriate, medication orders restrict patient deletion, order items cascade from their order, and medications referenced by historical order items cannot be deleted accidentally.

Monetary values use decimal precision, medication stock has a non-negative database check constraint, and seed data supplies roles, a development doctor, patients, and sample vital signs. The migration chain creates the initial schema, applies seed data, adds authentication state, introduces medication orders and stock fields, and synchronizes the final Sprint 1 model snapshot.

Migration review: The generated migration was inspected before application, and EF Core reports that no model changes remain outside the migration snapshot.

Key Relationship Decisions

- One Patient can have many VitalSigns, Medications, Appointments, and MedicationOrders.
- One MedicationOrder contains one or more MedicationOrderItems.
- Each MedicationOrderItem references exactly one Medication and stores a price snapshot.
- Deleting an order removes its items, while deleting a referenced medication is restricted.

Day 3: Catalog and Read Operations

The patient list became the sprint's primary catalog resource. GET /api/patients accepts pageNumber and pageSize, limits oversized pages, and returns the requested items together with total count and page metadata. This prevents an unbounded response as the database grows.

Optional `searchName` and `gender` parameters narrow the query only when supplied. `sortBy` supports first name, last name, or date of birth, and `isDescending` controls direction. The LINQ pipeline applies filters and ordering before `Skip` and `Take`, then projects directly to `PatientResponse` so the API does not expose EF Core entities or fetch unnecessary fields.

Example: GET

```
/api/patients?pageNumber=1&pageSize=10&searchName=a&gender=Female&sortBy=lastName&isDescending=true
```

Hands-on result: Implemented pagination, two optional filters, multiple sorting choices, DTO projection, authenticated access, Postman coverage, and HTTP-level tests for both authorized and unauthorized requests.

Day 4: Transactional Write Operations and Business Logic

Medication order creation goes beyond field mapping. The service first verifies that the patient exists and that every requested medication belongs to that patient. It rejects missing or inactive medications, checks all quantities before mutating state, and returns a conflict when stock is insufficient.

For a valid request, each order item captures `Quantity`, `UnitPrice`, and `LineTotal`, while the order stores the combined `TotalAmount`. Capturing `UnitPrice` on the order item preserves historical accuracy if the medication's current price changes later. Stock is decremented only after the complete request passes validation.

SQL Server operations run inside one EF Core transaction using `Serializable` isolation. The order header, order items, and stock changes therefore succeed or fail together, and an unexpected exception triggers rollback before the global middleware returns a safe `ProblemDetails` response.

Scenario	HTTP result	State change
Valid order	201 Created	Order saved and stock decremented
Patient or medication missing	404 Not Found	No changes
Inactive medication	409 Conflict	No changes
Insufficient stock	409 Conflict	No changes
Invalid request shape	400 Bad Request	Rejected by FluentValidation

Day 5: Sprint Review, Postman Demo, and Retrospective

Day 5 focused on evidence rather than another large feature. A dedicated Postman collection runs seven requests in order: doctor login, filtered catalog browsing, demo medication creation, stock inspection, successful order creation, stock-decrement verification, and insufficient-stock rejection. Collection variables automatically carry the JWT, medication ID, and expected stock between requests.

The Sprint Review checked status codes, happy and error paths, exception handling, migration state, documentation, and remaining security or concurrency risks. Incomplete external gates were not marked as finished: the live SQL Server demonstration, mentor approval, and merged pull-request link remain pending until real evidence exists.

Sprint 1 Implementation Snapshot

Area	Evidence	Review status
Schema and EF Core model	Entities, Fluent API, seed data, six migrations	Ready
Catalog endpoint	Pagination, filters, sorting, DTO projection	Tested
Medication order	Stock rules, totals, transaction, validation	Tested
Automated quality	27 passing unit and integration tests	Passed locally
Postman demonstration	Seven ordered requests with assertions	Ready for live run
Mentor review and merge	Requires external reviewer and GitHub PR	Pending

Required Deliverables

- Sprint 1 goal statement, sized backlog, and documented business-domain ERD.
- EF Core data model with explicit relationships, seed data, constraints, and current migrations.
- Paginated, filterable, sortable catalog endpoint that projects to response DTOs.
- Medication-order endpoint with stock checks, calculated totals, and one transaction boundary.
- Repeatable Postman demo plus happy-path and error-path automated coverage.
- Written Sprint Review, iteration backlog, retrospective, and one concrete Sprint 2 action.

Sprint 2 Backlog Priorities

1. Move JWT signing material out of tracked configuration and into secure secret storage.
2. Prevent public registration from selecting privileged roles.
3. Run a real SQL Server concurrency test for two orders competing for the final stock unit.
4. Complete mentor review, address comments, merge the pull request, and record its URL.

Cardiac Monitor Domain ERD

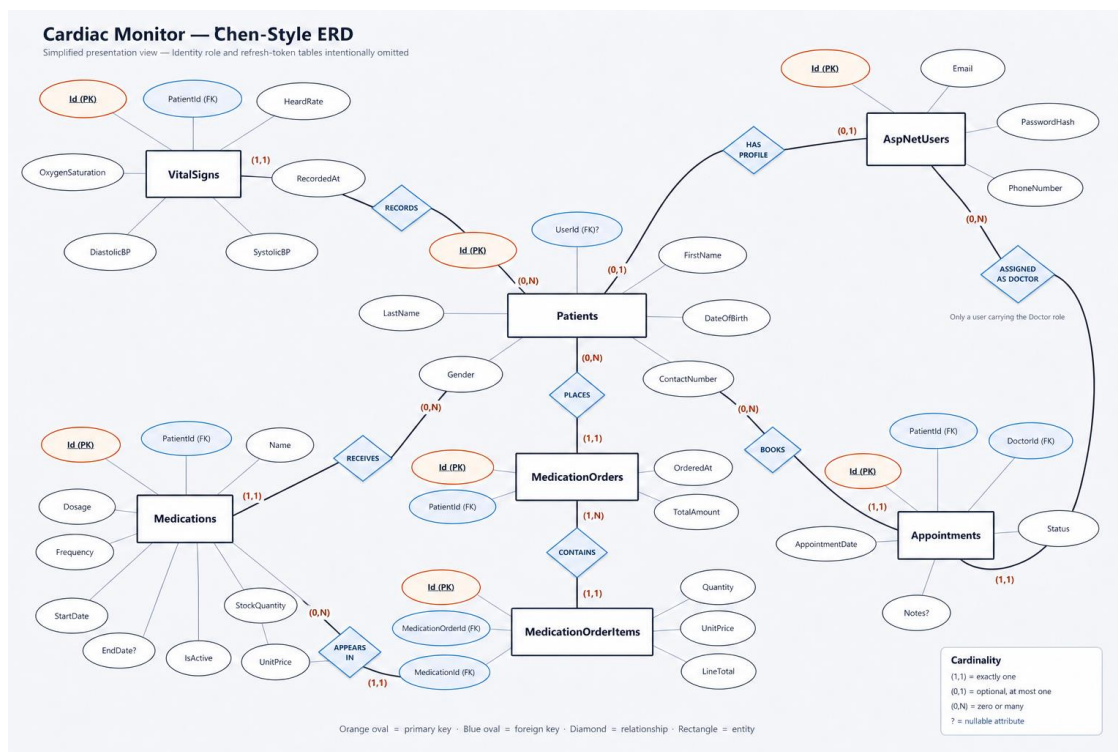


Figure 1. Business-domain entities, attributes, cardinalities, medication stock, and order relationships.

Testing and Mentor Check-in Checklist

1. Run dotnet test .\CardiacMonitor.slnx --configuration Release and confirm all 27 tests pass.
2. Run the Day 5 Postman collection against SQL Server and confirm all seven assertions are green.
3. Show the medication row before and after order creation to prove the stock decrement.
4. Confirm dotnet ef migrations has-pending-model-changes reports no pending model differences.
5. Present the ERD, migration history, Sprint Review, backlog, and retrospective.
6. Add the approved and merged pull-request URL to the Sprint 1 close-out summary.

Sprint 1 Retrospective

What went well: The project kept controllers thin, moved business logic into services, projected read queries to DTOs, used explicit EF Core relationships, and added testable success and error behavior around the most important sprint flows.

What should improve: Postman examples and review evidence should be maintained during implementation rather than assembled at the end. Security debt needs named owners, and InMemory tests must be complemented by relational concurrency coverage.

Concrete Sprint 2 action: Before implementing each endpoint, create its Postman example and its happy-path and error-path acceptance tests; the backlog card cannot move to Done until all three pass.